

FunkSpace Animation Manifest & Architecture

SVG-First Approach: FunkSpace animations should begin as SVG-based graphics to leverage crisp scaling and React's DOM integration ¹ ². In SVG, each shape is a DOM element that can be styled with CSS classes (e.g. Tailwind `fill-`/`stroke-` utilities) and have event listeners attached ². This means animations defined as SVG can use standard CSS and React props for theming. Crucially, text inside SVG remains actual text, making it searchable and screen-reader accessible ³. These features fit FunkSpace's goals of interactivity, theming, and accessibility.

Scalability & Performance (Canvas Fallback): SVG works well for most UI or demo animations, but it can slow down with thousands of objects or very high update rates ⁴ ⁵. For heavier or pixel-intensive motion (games, particle effects, large datasets), introduce a Canvas-based backend. As Tapflare notes, for "rich animations or pixel-based game graphics" you should use `<canvas>` or a library like PixiJS ⁵. In practice, the animator library can expose the same API but render onto a `<canvas>` behind the scenes when performance is critical. This staged strategy lets FunkSpace start simple with SVG and switch to Canvas modules as needed.

3D and Ultimate Performance (WebGL): If future experiments require true 3D, physics, VR or massive real-time scenes, the system should pivot to WebGL (e.g. via Three.js or React Three Fiber). WebGL "absolutely outperforms" SVG/Canvas at rendering thousands of shapes smoothly ⁶. It is warranted when 2D approaches hit their limits. The animator library can be designed so that each animation module has an abstract renderer: by default it emits SVG elements, but it could optionally draw the same animation in a WebGL scene. This hybrid approach (SVG → Canvas → WebGL) matches expert advice ⁵ ⁶.

Animation Manifest Structure: We will maintain a JSON *animation manifest* listing all supported animations and their metadata. Include a `lastUpdated` timestamp so the assistant knows when it was generated. Each animation entry can have fields like `id`, `description`, and its default `mode` (e.g. `"SVG"`). It can also include parameters (duration, easing, etc.) and a note if a Canvas/WebGL version exists. For example:

```
{
  "lastUpdated": "2025-11-04T10:33:11Z",
  "animations": [
    {
      "id": "spin",
      "description": "Continuously rotates an element around its center",
      "mode": "SVG",
      "parameters": { "duration": 2, "repeat": "infinite" }
    },
    {
      "id": "fadeIn",
      "description": "Fades an element in from transparent",
```

```

    "mode": "SVG",
    "parameters": { "duration": 1 }
  },
  {
    "id": "morphShape",
    "description": "Morphs one SVG path into another shape",
    "mode": "SVG",
    "parameters": { "duration": 3, "easing": "ease-in-out" }
  }
]
}

```

- `lastUpdated` : ISO timestamp of when the manifest was generated.
- `id` : Unique name for the animation.
- `description` : Human-readable summary of the animation's effect.
- `mode` : Default rendering mode (initially "SVG"; may later add "Canvas" or "WebGL" backends).
- `parameters` : Key animation settings (duration, easing, loops, etc.).

The assistant can use this manifest to know which animations exist and how to run them. It should always update the `lastUpdated` field when the manifest changes. By starting with SVG-based animations (fulfilling FunkSpace's React/Tailwind design goals) and outlining Canvas/WebGL options, this manifest and architecture plan meet current needs while allowing future growth ¹ ² .

Sources: FunkSpace's animation strategy aligns with expert guidance. SVG is recommended for sharp, DOM-driven graphics ¹ ² , Canvas/PixiJS for heavy 2D animation ⁵ , and WebGL (Three.js) for maximum 3D performance ⁶ . The above manifest structure is a suggested starting point that the assistant can always reference for planning animations.

¹ ² ³ ⁴ ⁵ ⁶ Comparing Web Graphics: Canvas, SVG, WebGL, and CSS | Tapflare
<https://tapflare.com/articles/web-graphics-comparison-canvas-svg-webgl>